Automates Avancés et Applications

Auteur
Yago Iglesias

24 octobre 2024

Table des matières

1 Introduction **1**

2 Langages Rationnels **2**
2.1 Définitions de base . . . 2
2.2 Langages rationnels : définitions . . . 3

3 Automates finis **4**
3.1 Automates finis déterministes . . . 4
3.2 Automates finis non déterministes + ε-transitions . . . 5
3.3 Déterminisation d'un AFN . . . 6
3.4 Équivalence entre expressions rationnelles et automates finis déterministes . . . 6

4 Minimisation d'automates **8**
4.1 Morphismes d'automates . . . 8
4.2 Minimisation d'automates . . . 10
4.2.1 Algorithme de Brzozowski . . . 10
4.2.2 Algorithme de Moore . . . 11

5 Monoïdes **12**
5.1 Reconnaissance par monoïdes . . . 14

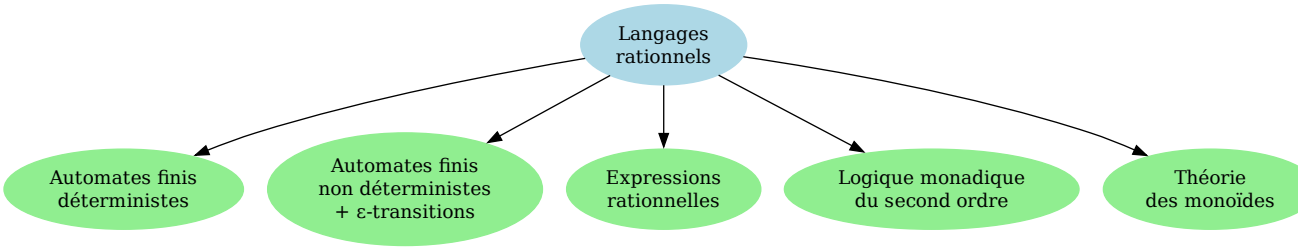
6 Logique monadique du second ordre (MSO) **17**
6.1 Syntaxe . . . 18
6.2 Sémantique . . . 18
6.2.1 Sémantique de la logique monadique du premier ordre . . . 18
6.2.2 Sémantique de la logique monadique du second ordre . . . 19
6.3 Relation avec les expressions rationnelles . . . 20
6.3.1 Éléments idempotents . . . 22
6.3.2 Relations de Green . . . 22

1 Introduction

Ce document est un recueil de notes du cours d'Automates Avancés et Applications de niveau M1. Il est basé sur les cours de Mme. Daniela Petrisan et M. Roberto Mantaci à Université Paris Cité, cependant toute erreur ou inexactitude est de ma responsabilité.

Ce document a été rédigé principalement par Yago Iglesias, mais tout contributeur peut être retrouvé dans la section contributeurs du répertoire GitHub. Un remerciement particulier est adressé à Erin Le Boulc'h pour sa participation active à la rédaction et correction de ce document.

2 Langages Rationnels



2.1 Définitions de base

Définition 2.1 (alphabet)**.** Un **alphabet** est un ensemble fini de lettres ou de symboles.

Définition 2.2 (mot)**.** Un **mot** sur un alphabet Σ est une séquence de lettres de Σ. On écrit $w = w_1 w_2 \dots w_n$ où $w_i \in \Sigma, \forall i \in \{1, \dots, n\}$. La longueur d'un mot w est notée $|w| = n$, pour $w = w_1 w_2 \dots w_n$.

Notation 2.1. On note ε le mot vide et Σ^* l'ensemble des mots sur Σ.

Définition 2.3 (concaténation)**.** La **concaténation** de deux mots w et v est notée wv. Si $w = w_1 w_2 \dots w_n$ et $v = v_1 v_2 \dots v_m$, alors $wv = w_1 w_2 \dots w_n v_1 v_2 \dots v_m$.

Définition 2.4 (langage)**.** Un langage sur un alphabet Σ est un sous-ensemble de Σ^*.

Définition 2.5 (concaténation de langages)**.** Soient L_1, L_2 deux langages sur Σ, leur **concaténation** est le langage

$$L_1 L_2 = \{w_1 w_2 \mid w_1 \in L_1, w_2 \in L_2\}$$

Exemple 2.1.1.

$$\Sigma = \{a, b\}, \quad L = \{a, ab, bb\}, \quad K = \{b, ab\}$$

$$LK = \{ab, abb, bbb, aab, abab, bbab\}$$

Définition 2.6 (étoile de Kleene)**.** Soit L un langage sur Σ, son **étoile de Kleene** est le langage

$$L^* = \bigcup_{n \in \mathbb{N}} L^n$$

où $L^0 = \{\varepsilon\}$ et $L^{n+1} = LL^n$.

Remarque 2.1.1. *Est-ce que $L^* = \{w^n \mid w \in L, n \in \mathbb{N}\}$?*
Non, on peut trouver un contre-exemple avec $L = \{a, b\}$.
On a bien que $ab \in L^*$, mais $ab \notin \{w^n \mid w \in L, n \in \mathbb{N}\} = \{a^n \mid n \in \mathbb{N}\} \cup \{b^n \mid n \in \mathbb{N}\}$.

Remarque 2.1.2. *Est-ce que* $L(M \cap N) = LM \cap LN$ *?*
Non, on peut trouver un contre-exemple avec $L = \{a, ab\}$*,* $M = \{b\}$ *et* $N = \{\varepsilon\}$*.*
On a que $M \cap N = \emptyset$*, donc* $L(M \cap N) = \emptyset$*.*
Mais $LM = \{ab, abb\}$ *et* $LN = \{a, ab\}$*, donc* $LM \cap LN = \{ab\} \neq \emptyset$*.*

Exercice 2.1.1. Montrer que la concaténation de langages est distributive par rapport à l'union, *i.e.* que $L(M \cup N) = LM \cup LN, \forall L, M, N$ langages.

Démonstration.

$$\begin{aligned}
w \in L(M \cup N) &\iff \exists w_L \in L, \exists w_{M\cup N} \in M \cup N, w = w_L w_{M\cup N} \\
&\iff \exists w_L \in L, (\exists w_M \in M, w = w_L w_M \vee \exists w_N \in N, w = w_L w_N) \\
&\iff (\exists w_L \in L, \exists w_M \in M, w = w_L w_M) \vee (\exists w_L \in L, \exists w_N \in N, w = w_L w_N) \\
&\iff w \in LM \vee w \in LN \\
&\iff w \in (LM \cup LN)
\end{aligned}$$

□

2.2 Langages rationnels : définitions

Définition 2.7 (langage rationnel)**.** Soit Σ un alphabet fini, l'ensemble ERat des expressions rationnelles sur Σ est défini comme suit :
- $\varepsilon \in ERat$
- $\emptyset \in ERat$
- $\forall a \in \Sigma, a \in ERat$
- $\forall E, F \in ERat, E + F \in ERat$
- $\forall E, F \in ERat, EF \in ERat$
- $\forall E \in ERat, E^* \in ERat$

Définition 2.8 (sémantique des expressions rationnelles)**.** Soit $r \in$ ERat, on définit le langage $\mathcal{L}(r)$ associé à r par induction sur la structure de r :
- $\mathcal{L}(\varepsilon) = \{\varepsilon\}$
- $\mathcal{L}(\emptyset) = \emptyset$
- $\mathcal{L}(a) = \{a\}$
- $\mathcal{L}(E + F) = \mathcal{L}(E) \cup \mathcal{L}(F)$
- $\mathcal{L}(EF) = \mathcal{L}(E)\mathcal{L}(F)$
- $\mathcal{L}(E^*) = \mathcal{L}(E)^*$

Exemple 2.2.1. $\Sigma = \{a, b\}, \quad r = (a + b)^* a \in$ ERat, alors $\mathcal{L}(r) = \{wa \mid w \in \Sigma^*\}$

Définition 2.9 (langage rationnel)**.** Un langage L sur un alphabet Σ est dit **rationnel** s'il existe une expression rationnelle $r \in$ ERat telle que $L = \mathcal{L}(r)$.

Exemple 2.2.2. $\{a^n \mid n \in \mathbb{N}\}$ est un langage rationnel engendré par l'expression rationnelle a^*.
Cependant, $\{a^n b^n \mid n \in \mathbb{N}\}$ n'est pas un langage rationnel.

3 Automates finis

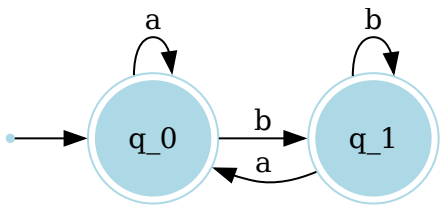
3.1 Automates finis déterministes

Définition 3.1 (automate fini déterministe)**.** Soit Σ un alphabet, un **automate fini déterministe** (AFD) est un tuple $\langle Q, q_0, F, \delta \rangle$ où

- Q est un ensemble fini d'états
- $q_0 \in Q$ est appelé l'état initial
- $F \subseteq Q$ est l'ensemble des états finaux / acceptants
- $\delta : Q \times \Sigma \to Q$ est la fonction de transition

Exemple 3.1.1. Un automate fini déterministe :

- $\Sigma = \{a, b\}$
- $Q = \{q_0, q_1\}$
- q_0 est l'état initial
- $F = \{q_0, q_1\}$
- $\delta : \begin{cases} (q_0, a) & \mapsto q_0 \\ (q_0, b) & \mapsto q_1 \\ (q_1, a) & \mapsto q_0 \\ (q_1, b) & \mapsto q_1 \end{cases}$



Définition 3.2 (lecture d'un mot par un AFD)**.** Soit $A = \langle Q, q_0, F, \delta \rangle$ un AFD qui a pour alphabet Σ. On définit la fonction δ^* par induction sur la longueur du mot w :

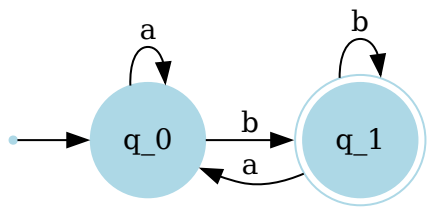
$$\begin{array}{rcl} \delta^* : Q \times \Sigma^* & \to & Q \\ (q, \varepsilon) & \mapsto & q \\ (q, wa) & \mapsto & \delta(\delta^*(q, w), a) \end{array}$$

On a alors que $\delta^*(q, w)$ est l'état atteint par A après avoir lu le mot w depuis l'état q.

Définition 3.3 (langage reconnu par un AFD)**.** Le langage reconnu / accepté par un AFD $A = \langle Q, q_0, F, \delta \rangle$ est le langage

$$\mathcal{L}(A) = \{w \mid w \in \Sigma^*, \delta^*(q_0, w) \in F\}$$

Exemple 3.1.2. Pour l'automate A suivant, avec $\Sigma = \{a, b\}$, le langage reconnu est $\mathcal{L}(A) = \mathcal{L}((a+b)^*b) = \mathcal{L}((a+b)^*bb^*) = \{wb \mid w \in \Sigma^*\}$.



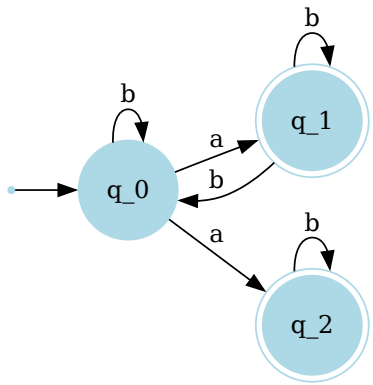
3.2 Automates finis non déterministes + ε-transitions

Dans cette partie, on traite en meme temps les cas avec et sans ε-transitions. Les additions en vert correspondent à la définitions avec ε-transitions. En ignorant cela, on retrouve la definition d'un automate fini non déterministe.

Définition 3.4 (automate fini non déterministe + ε-transitions)**.** Soit Σ un alphabet, un **automate fini non déterministe** (AFN) est un tuple $\langle Q, I, F, \delta \rangle$ où

- Q est un ensemble fini d'états
- $I \subseteq Q$ est l'ensemble des états initiaux
- $F \subseteq Q$ est l'ensemble des états finaux / acceptants
- $\delta : Q \times (\Sigma \cup \{\varepsilon\}) \to \mathcal{P}(Q)$ est la fonction de transition

Exemple 3.2.1. Un automate fini non déterministe avec $\Sigma = \{a, b\}$:



Définition 3.5 (lecture d'un mot par un AFN)**.** Soit $A = \langle Q, I, F, \delta \rangle$ un AFN qui a pour alphabet

Σ. On définit la fonction δ^* par induction sur la longueur du mot w :

$$\begin{array}{rcl} \delta^* : Q \times \Sigma^* & \to & \mathcal{P}(Q) \\ (q, \varepsilon) & \mapsto & \{q\} \\ (q, wa) & \mapsto & \bigcup\limits_{p \in \delta^*(q,w)} \delta(p, a) \end{array}$$

Définition 3.6 (langage reconnu par un AFN)**.** Le langage reconnu / accepté par un AFN $A = \langle Q, I, F, \delta \rangle$ est le langage

$$\mathcal{L}(A) = \{w \in \Sigma^* \mid \exists q_0 \in I, \delta^*(q_0, w) \cap F \neq \emptyset\}$$

3.3 Déterminisation d'un AFN

Soit $A = \langle Q, I, F, \delta \rangle$ un AFN, on considère l'AFD $A' = \langle \mathcal{P}(Q), I, F', \delta' \rangle$ où

— $F' = \{q \in \mathcal{P}(Q) \mid q \cap F \neq \emptyset\}$

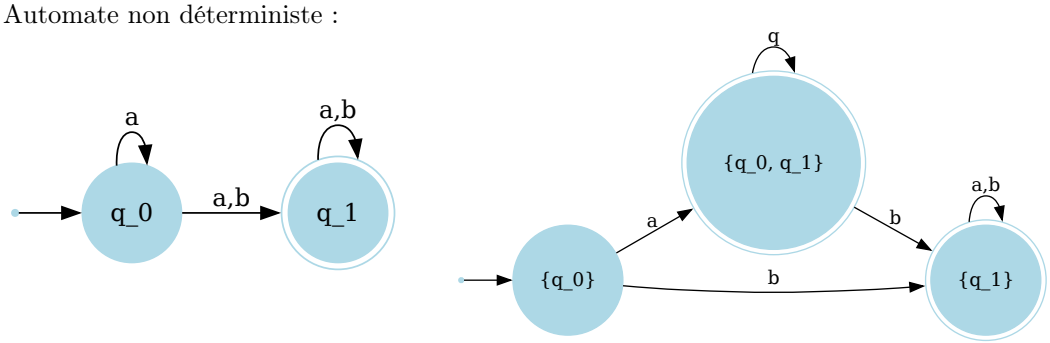
— $$\begin{array}{rcl} \delta' : \mathcal{P}(Q) \times \Sigma & \to & \mathcal{P}(Q) \\ (Q, a) & \mapsto & \bigcup\limits_{p \in Q} \delta(p, a) \end{array}$$

Ce processus est appelé **déterminisation** d'un AFN et nous permet de transformer un AFN en un AFD équivalent.

Exemple 3.3.1. Déterminisation d'un automate non déterministe :

Automate déterminisé :

Automate non déterministe :



Théorème 3.1. *Soit A un automate fini non déterministe avec des ε-transitions, alors il existe un automate fini non déterministe A', tel que $\mathcal{L}(A) = \mathcal{L}(A')$.*

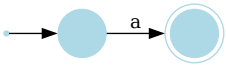
3.4 Équivalence entre expressions rationnelles et automates finis déterministes

Théorème 3.2 (Injection des expressions rationnelles vers les automates finis déterministes)**.** *Soit $r \in ERat$, alors il existe un automate fini déterministe $N(r)$ tel que $\mathcal{L}(A) = \mathcal{L}(r)$.*

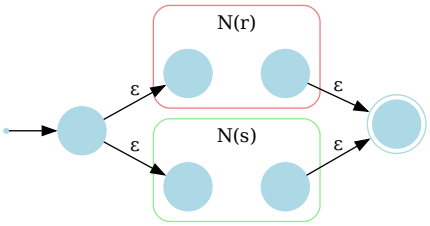
Démonstration. Nous allons construire un tel automate par induction sur la structure de r cependant la preuve du fait que cet automate reconnaît le langage associé à r est omise. Cette construction est appelée **construction de Thompson**.

$N(\varepsilon)$

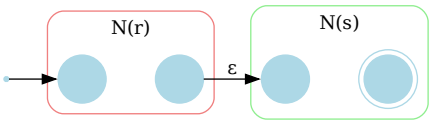
$N(\emptyset)$



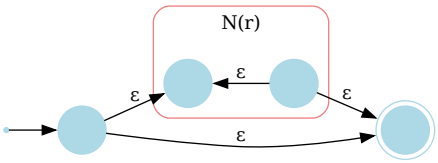
$N(a), a \in \Sigma$



$N(r+s)$



$N(rs)$



$N(r^*)$

En combinant ces constructions, on peut construire un automate fini non déterministe pour n'importe quelle expression rationnelle qui peut être determinisé pour obtenir un automate fini déterministe.

□

4 Minimisation d'automates

4.1 Morphismes d'automates

Définition 4.1 (Morphisme d'automates)**.** Soit $A = \langle Q, q_0, F, \delta \rangle$ et $A' = \langle Q', q'_0, F', \delta' \rangle$ sur un alphabet Σ. Alors un morphisme d'automates $\varphi : A \to A'$ est une fonction $\varphi : Q \to Q'$ telle que

1. $\varphi(q_0) = q'_0$
2. $\forall q \in Q, \quad \varphi(q) \in F' \iff q \in F$
3. $\forall q \in Q, \forall a \in \Sigma, \quad \delta'(\varphi(q), a) = \varphi(\delta(q, a))$ *i.e.* $\varphi \circ \delta_a = \delta'_a \circ \varphi$

$$\begin{tikzcd} Q \arrow[r, "\delta_a"] \arrow[d, "\varphi"'] & Q \arrow[d, "\varphi"] \\ Q' \arrow[r, "\delta_a"] & Q' \end{tikzcd}$$

Exercice 4.1.1. Soit $\varphi : A \to A'$ un morphisme d'automates déterministes, alors $\mathcal{L}(A) = \mathcal{L}(A')$

Démonstration.

$$\begin{aligned} w \in \mathcal{L}(A) \quad &\iff \quad \delta^*(q_0, w) \in F \\ &\iff \quad \varphi(\delta^*(q_0, w)) \in F' \quad (\textit{par } 1) \\ &\iff \quad \delta^*(\varphi(q_0), w) \in F' \quad (\textit{par } 3) \\ &\iff \quad \delta^*(q'_0, w) \in F' \quad (\textit{par } 2) \\ &\iff \quad w \in \mathcal{L}(A') \end{aligned}$$

□

Définition 4.2 (Quotient d'un automate)**.** Soit $L \subseteq \Sigma^*$ et $w \in \Sigma^*$. Le quotient $w^{-1}L$ est défini par

$$w^{-1}L = \{u \in \Sigma^* \mid wu \in L\}$$

Définition 4.3. Soit A $= \langle Q, q_0, F, \delta \rangle$ sur l'alphabet Σ et $q \in Q$ alors on définit

$$L_q = \{w \in \Sigma^* \mid \delta^*(q, w) \in F\}$$

Lemme 4.1. *Soit $A = \langle Q, q_0, F, \delta \rangle$ un automate fini déterministe, $q \in Q$ et $w \in \Sigma^*$. Si $\delta^*(q_0, w) = q$ alors $L_q = w^{-1}\mathcal{L}(A)$*

Démonstration.

$$\begin{aligned} u \in L_q \quad &\iff \quad \delta^*(q, u) \in F \\ &\iff \quad \delta^*(\delta^*(q_0, w), u) \in F \\ &\iff \quad \delta^*(q_0, wu) \in F \\ &\iff \quad wu \in \mathcal{L}(A) \\ &\iff \quad u \in w^{-1}\mathcal{L}(A) \end{aligned}$$

□

Corollaire 4.1. *Si $L \subseteq \Sigma^*$ est un langage régulier alors l'ensemble $\left\{w^{-1}L \mid w \in \Sigma^*\right\}$ est fini.*

Démonstration. Si $L = \mathcal{L}(A)$, avec A $= \langle Q, q_0, F, \delta\rangle$, alors $\left|\left\{w^{-1}L \mid w \in \Sigma^*\right\}\right| \leq |Q|$. Car $\forall w \in \Sigma^*$, $w^{-1}L$ est le langage accepté par A à partir de l'état $\delta^*(q_0, w)$. □

Corollaire 4.2. *Si $\langle Q, I, F, \delta\rangle$ accepte un langage L alors $|Q| \geq \#$ quotients de L*

Définition 4.4 (Automate des quotients)**.** Soit $L \subseteq \Sigma^*$ un langage régulier, soit $Q = \left\{w^{-1}L \mid w \in \Sigma^*\right\}$ et $\langle Q, q_0, F, \delta\rangle$. Alors l'automate des quotients de L est défini par
- L'état initial est $\varepsilon^{-1}L$
- $F = \left\{w^{-1}L \mid w \in L\right\}$
- $\delta(w^{-1}L, a) = (wa)^{-1}L$

Remarque 4.1.1. *Si $w^{-1}L = (w')^{-1}L$, alors $(wa)^{-1}L = (w'a)^{-1}L$.*
Soit $w, w' \in \Sigma^*$ tel que $w^{-1}L = (w')^{-1}L$. Soit $a \in \Sigma$ et $u \in \Sigma^*$,

$$\begin{aligned} u \in (wa)^{-1}L &\iff wau \in L \\ &\iff au \in w^{-1}L \\ &\iff au \in w'^{-1}L \\ &\iff w'au \in L \\ &\iff (w'a)u \in L \\ &\iff u \in (w'a)^{-1}L \end{aligned}$$

Donc la fonction de transition δ de l'automate des quotients est bien définie.

Proposition 4.1. *Étant donné un automate A qui accepte le langage L, l'automate des quotients accepte aussi L et il est minimal parmi les automates acceptant L.*

Démonstration. $\delta^*(L, w) = w^{-1}L$ et donc $\underbrace{\delta^*(L, w) \in F}_{w \in \mathcal{L}(A)} \iff w \in L$

Et ainsi, par le corollaire 4.2, l'automate des quotients est minimal. □

Lemme 4.2. *Soit $L \subseteq \Sigma^*$ un langage régulier. Soit A_L l'automate des quotients de L et soit B un autre automate acceptant L. Soit reach(B) un sous-automate accessible de B. Alors on a un morphisme surjectif (un quotient) d'automates reach$(B) \twoheadrightarrow A_L$.*

$$\begin{tikzcd} & reach(B) \arrow[dl, two heads] \arrow[dr, hook] & \\ A_L & & B \end{tikzcd}$$

Si $B =< F, t_0, F, \delta >$ alors reach$(B) =< F', t_0, F', \delta' >$ où
- $F' = \{t \in F \mid \exists w \in \Sigma^*, \delta^*(t_0, w) = t\}$
- $F' = F \cap F'$
- $\delta' = \delta \cap (F' \times \Sigma \times F')$

Exercice 4.1.2. On définit $\varphi : \text{reach}(B) \to A$.
Si $t \in F', \exists w \in \Sigma^*$ tel que $t = \delta^*(t_0, w)$ On définit $\varphi(t) = w^{-1}L$
Montrer que φ est bien définie, *i.e.* , $\varphi(t) = w'^{-1}L \implies w^{-1}L = w'^{-1}L$.

Démonstration. Vérifions φ surjective.

Si $w \in \Sigma^*$ et $w^{-1}L$ est un état de A_L alors $\varphi(\delta^*(t_0, w)) = w^{-1}L$

Si $|B| = |A_L|$, alors B est un automate accesible (Sinon $|A| \leq |\text{reach}(B)| < |B| \lightning$).

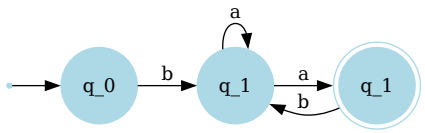
Donc $B = \text{reach}(B)$

De plus, le morphisme φ est une bijection et φ^{-1} est un morphisme d'automates. Donc $B \cong A_L$. □

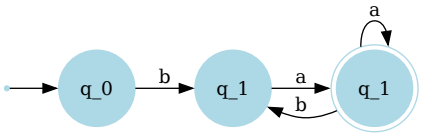
4.2 Minimisation d'automates

Rappel 4.2.1. *Pour tout langage rationnel L, il existe un unique (à isomorphisme près) automate déterministe minimal (avec le plus petit nombre possible d'états) qui reconnait L.*

Remarque 4.2.1. *Ce n'est pas vrai pour les automates non-déterministes.*



$(ba^+)^+$



$(ba^+)^+$

L'automate des résiduels de L est l'automate minimal reconnaissant L.

Nous allons voir trois algorithmes de minimisation différents :

— Algorithme de Brzozowski
— Algorithme de Moore
— Algorithme de Hopcroft

4.2.1 Algorithme de Brzozowski

Proposition 4.2. *Si A est un automate déterministe et accesible, i.e. , $\forall q \in Q, \exists$ un chemin $q_0 \to q$, et $A^\sim = det(mirr(A))$ est l'automate obtenu en déterminisant l'automate miroir de A. Alors $A^\sim$ est l'automate minimal qui reconnait $\widetilde{\mathcal{L}(A)}$.*

Démonstration. Soit $A = (Q, q_0, F)$, $L = \mathcal{L}(A)$. $\text{mirr}(A) = (Q, F, q_0), M = \mathcal{L}(\text{mirr}(A)) = \widetilde{L}$.

On note $X \cdot u$ et $\delta(X, u)$, X étant un ensemble d'états et u un mot, l'ensemble $\bigcup_{q \in X} \delta(q, u)$.

Nous allons montrer que $A^\sim$ est l'automate minimal pour M. Pour cela, il suffit de montrer que $A^\sim$ est isomorphe à l'automate des résiduels de M.

Pour cela, il faut montrer que si u et v sont deux mots quelconques tel que $u^{-1}M = v^{-1}M$, alors $F \cdot u = F \cdot v$ dans $A^\sim$.

Soit p un état de $F \cdot u$. Puisque A est accesible, il existe un chemin de $q_0 \to p$ dans A, et donc il existe un chemin $p \to q_0$ dans $\text{mirr}(A)$. Soit w l'étiquette de ce chemin, alors le mot uw est l'étiquette d'un chemin réussi de $\text{mirr}(A)$. En conséquence, $uw \in M$.

Mais $uw \in M \iff w \in u^{-1}M \iff w \in v^{-1}M \iff vw \in M$.

Donc vw est l'étiquette d'un chemin réussi Γ dans $\mathrm{mirr}(A)$. Étant donné que A est déterministe, tout chemin aboutissant dans q_0 et dont l'étiquette a w comme suffixe doit passer par p.

On en déduit que $p \in Tv$. □

Corollaire 4.3. *Soit A un automate, alors l'automate det(mirr(det(mirr(A)))) est l'automate minimal pour $\mathcal{L}(A)$.*

Démonstration. En effet,

$$\underbrace{\det(\mathrm{mirr}(\underbrace{\det(\mathrm{mirr}(A))}_{\text{Un automate déterministe et accesible qui reconnait } \widetilde{\mathcal{L}(A)}}))}_{\text{la proposition précédente garantit que cet automate est minimal pour } \widetilde{\widetilde{\mathcal{L}(A)}}=\mathcal{L}(A)}$$ □

Complexité. *La complexité de l'algorithme est en $O(2^n)$. En effet, la dernière détermination exécutée travaille sur un automate de taille au plus 2^n et produit un automate de taille 2^n. Sa complexité reste aussi en $O(2^n)$.*

Remarque 4.2.2. *Alors que d'autres algorithmes, tel que Moore, ont une complexité polynomiale, Brzozowski à différence des autres, ne nécessite pas que l'automate donné soit déterministe.*

4.2.2 Algorithme de Moore

Définition 4.5 (Congruence d'automates)**.** Soit $A = (Q, q_0, F, S)$ un automate déterministe et soit $\sim$ une relation d'équivalence définie sur l'ensemble Q. On dit que $\sim$ est une congruence si $\sim$ satisfait les conditions suivantes :

1. Compatibilité aves les transitions : Si $q \sim q'$ alors $\forall a \in \Sigma, \delta(q, a) \sim \delta(q', a)$
2. Saturation de A : Si $q \sim q'$ alors $q \in F \iff q' \in F$

Définition 4.6. Si $A = \langle Q, q_0, F, \delta \rangle$ est un AFD et $\sim$ une congruence définie sur Q. On définit l'automate quotient :

$$A/\sim = (Q', q_0', F', \delta')$$

avec
— $Q' = \{[q] \mid q \in Q\}$ (chaque état est étiqueté par une classe d'équivalence)
— $q_0' = [q_0]$
— $F' = \{[q] \mid q \in F\}$
— $\delta'([q], a) = [p]$ si et seulement si $p \in [qa]$

Proposition 4.3. *Si A est un automate déterministe et $\sim$ une congruence sur A, alors $\mathcal{L}(A) = \mathcal{L}(A/\sim)$.*

Définition 4.7 (Congruence de Nerode)**.** Si $A = (Q, \{q_0\}, F, \delta)$ un AFD, $\forall q, q' \in Q$ on définit $q \cong q' \iff L_q = L_{q'}$.

Rappel 4.2.2. $L_q = \{w \mid \delta^*(q, w) \in F\}$. *Donc*

$$\begin{aligned} q \cong q' \iff L_q = L_{q'} &\iff \forall w \in \Sigma^*, w \in L_q \textit{ si et seulement si } w \in L_{q'} \\ &\iff \forall w, \delta^*(q, w) \in F \textit{ si et seulement si } \delta^*(q', w) \in F \end{aligned}$$

On en déduit que

$$q \ncong q' \iff \exists w, \delta^*(q,w) \in F \text{ et } \delta^*(q',w) \notin F \tag{1}$$

où $\delta^*(q,w) \notin F$ *et* $\delta^*(q',w) \in F$ *sont deux états non équivalents, dits* séparables.

Si q et q' sont séparables et w est un mot qui satisfait 1, on dira que w *sépare* q et q'.
Pour calculer la congruence de Nérode on introduit une famille de congruences :

$$\cong_i, i \in \mathbb{N}, q \cong_i q' \iff \forall w, |w| \leq i \implies \delta^*(q,w) \in F \iff \delta^*(q',w) \in F$$

Définition 4.8. Définition alternative
— $q \cong_0 q'$ si et seulement si $q \in F \iff q' \in F$
— $q \cong_{i+1} q'$ si et seulement si $q \cong_i q'$ et $\forall a \in \Sigma, \delta(q,a) \cong_i \delta(q',a)$

Remarque 4.2.3. *Par définition* $\cong_{i+1}$ *induit une partition de* Q *au moins aussi fine que celle induite par* $\cong_i$.
Les classes de $\cong_{i+1}$ *sont obtenues en partitionnant des classes de* $\cong_i$.

Remarque 4.2.4. *Puisque* Q *est fini, le processus de partitionnement doit se stabiliser. Autrement dit,*

$$\exists k \in \mathbb{N}^*, \textit{tel que } \cong_k = \cong_{k+j} \forall j \in \mathbb{N}^*$$

Proposition 4.4. *Si* $k \in \mathbb{N}$ *tel que* $\cong_k = \cong_{k+j} \forall j \in \mathbb{N}$, *alors* $\cong_k = \cong$

Démonstration. Soit $q, q' \in Q$, $q \cong_k q'$, montrer que $L_q = L_{q'}$.
Soit $w \in L_q$.
— Si $|w| \leq k$ alors $\delta^*(q',w) \in F$ car $q \cong_k q'$
— Si $|w| > k$ alors $\exists i$ tel que $|w| = k+i$ et donc comme $\cong_k = \cong_{k+i}$ on a que $\delta^*(q',w) \in F$
Et donc $w \in L_{q'}$ et ainsi $L_q \subseteq L_{q'}$. On peut faire la meme preuve pour montrer que $L_{q'} \subseteq L_q$. On en déduit que que $L_q = L_{q'}$. □

Définition 4.9 (Algorithme de Moore)**.** L'algorithme consiste à créer un automate à partir des classes d'équivalence sur $\cong$. Les états sont les classes d'équivalence, on obtient les transitions en regardant le comportement d'un représentant de la classe et les états finaux sont les classes qui ont un représentant qui est un état final.

Complexité. *L'algorithme effectue* n *étapes et chaque étape dépense* $O(n)$, *donc sa complexité est en* $O(n^2)$ *(au pire).*
En fait, en moyenne $O(n \log n)$ *et même* $O(n \log \log n)$, *[David(2010)]*

5 Monoïdes

Définition 5.1 (Monoïde)**.** Un monoïde est un tuple $(M, \times, 1)$ où M est un ensemble, $\times : M \times M \to M$ est une operation binaire sur M, $1 \in M$ tel que

1. $\times$ est une opération associative ($\forall x, y, z \in M, x \times (y \times z) = (x \times y) \times z$)
2. 1 est un élément neutre pour $\times$, c'est-à-dire , $\forall x \in M, 1 \times x = x \times 1 = x$

Exemple 5.0.1. — $(\mathbb{N}, +, 0)$

— $(\Sigma^*, \cdot, \varepsilon)$
— $(\mathbb{Z}, +, 0)$ est un monoïde, mais aussi un groupe car tout élément possède un symétrique.
— Si Q est un ensemble, alors $(Q^Q, \circ, \mathrm{id})$

Définition 5.2 (Morphisme de monoïdes)**.** Soit $(M, \times_M, 1_M)$ et $(N, \times_N, 1_N)$ des monoïdes. Une fonction $h : M \to N$ est appelée un morphisme de monoïdes si et seulement si

1. $h(1_M) = 1_N$ (*h préserve l'élément neutre*)
2. $\forall x, y \in M, h(x \times_M y) = h(x) \times_N h(y)$ (*h préserve la multiplication*)

Proposition 5.1. *Soit Σ un ensemble et $(M, \cdot, 1)$ un monoïde. Soit $f : \Sigma \to M$ une fonction. Il existe un unique morphisme de monoïdes $\bar{f} : \Sigma^* \to M$ tel que $\bar{f}(a) = f(a), \forall a \in \Sigma$.*

$$\begin{tikzcd} \Sigma \arrow[r, hook] \arrow[d, "f"] & \Sigma^* \arrow[dl, dashed, "\exists! \bar{f}"] \\ M & \end{tikzcd}$$

Démonstration. — Existence :

Soit $w \in \Sigma^*$ de la forme $w = w_1 w_2 \dots w_n$ où $w_i \in \Sigma$.
Soit $\bar{f}(w) =^{def} f(w_1) \cdot \dots \cdot f(w_n)$ et $\bar{f}(\varepsilon) = 1_M$. Alors pour $a \in \Sigma, \bar{f}(a) = f(a)$.
Si $w = w_1 w_2 \dots w_n$ et $v = v_1 v_2 \dots v_m \in \Sigma^*$.

$$\begin{aligned} \bar{f}(wv) = \bar{f}(w_1 w_2 \dots w_n v_1 v_2 \dots v_m) &= f(w_1) \cdot \dots \cdot f(w_n) \cdot f(v_1) \cdot \dots \cdot f(v_m) \\ &= (f(w_1) \cdot \dots \cdot f(w_n)) \cdot (f(v_1) \cdot \dots \cdot f(v_m)) \\ &= \bar{f}(w_1 w_2 \dots w_n) \cdot \bar{f}(v_1 v_2 \dots v_m) \\ &= \bar{f}(w) \cdot \bar{f}(v) \end{aligned}$$

Donc $\forall w, v \in \Sigma^*, \bar{f}(wv) = \bar{f}(w)\bar{f}(v)$ et $\bar{f}(\varepsilon) = 1_M$ et ainsi, $\bar{f}$ est un morphisme de monoïdes.

— Unicité :

Si $h : \Sigma^* \to M$ est un morphisme de monoïdes tel que $h(a) = f(a), \forall a \in \Sigma^*$. Alors $h(\varepsilon) = 1_M$.

Comme h préserve la multiplication, on a $h(a_1 a_2 \dots a_n) = h(a_1) \dots h(a_n) = f(a_1) \dots f(a_n) = \bar{f}(a_1 a_2 \dots a_n)$.

Et donc $h = \bar{f}$

□

On dit que Σ^* est librement engendré sur Σ.

Définition 5.3 (Librement engendré)**.** H est librement engendré sur Σ si

$$\forall f : \Sigma \to M \text{ où } M \text{ est un monoïde}, \exists! \bar{f} : H \to M \text{ et } i : \Sigma \to H, \bar{f} \circ i = f$$

$$\begin{tikzcd} \Sigma \arrow[r, hook, "i"] \arrow[d, "f"] & H \arrow[dl, dashed, "\exists! \bar{f}"] \\ M & \end{tikzcd}$$

Et on a donc que $H \cong \Sigma^*$.

Exemple 5.0.2. Montrons que $(\mathbb{N}, +, 0)$ est librement engendré sur $\{1\}$.

$$\begin{tikzcd} \{1\} \arrow[r, hook] \arrow[d, "f"] & (\mathbb{N}, +, 0) \arrow[dl, dashed, "\exists! \bar{f}"] \\ (M, \cdot, 1_M) & \end{tikzcd}$$

Alors on a que $f(1) = m \in M$ et donc

$$f(n) = f(\underbrace{1 + \ldots + 1}_{n \text{ fois}}) = \underbrace{f(1) \cdot \ldots \cdot f(1)}_{n \text{ fois}} = \underbrace{m \cdot \ldots \cdot m}_{n \text{ fois}} = m^n$$

Donc, si on pose $i : \{1\} \to (\mathbb{N}, +, 0)$ tel que $f(1) = 1$, alors quelque soit le choix de m, *i.e.* , quelque soit la fonction f choisie (car elle dépend juste du choix m), alors on peut toujours poser $\bar{f}(\mathbb{N}, +, 0)$ avec $f(0) = 1_M$ et $f(1) = m$ qui vérifie bien que $\bar{f} \circ i = f$. Ainsi, $(\mathbb{N}, +, 0)$ est librement engendré sur $\{1\}$.

Remarque 5.0.1. $\mathbb{N} \cong \{1\}^*$

5.1 Reconnaissance par monoïdes

Définition 5.4 (Reconnaissance par monoïdes)**.** Soit Σ un alphabet fini. Soit $L \subseteq \Sigma^*$ un langage et soit $\varphi : \Sigma^* \to M$ un morphisme de monoïdes où $(M, \circ, 1_M)$ est un monoïde fini. On dit que φ reconnait L si et seulement si il existe $P \subseteq M$ tel que $L = \varphi^{-1}(P)$, c'est-à-dire $L = \{w \in \Sigma^* \mid \varphi(w) \in P\}$.

Exercice 5.1.1. φ reconnaît L si et seulement si $L = \varphi^{-1}(\varphi(L))$.

Démonstration.
- $\boxed{\Leftarrow}$
  Soit $P = \varphi(L) \subseteq M$, alors on a que $\varphi^{-1}(P) = \varphi^{-1}(\varphi(L)) = L$ et donc φ reconnait L.
- $\boxed{\Rightarrow}$
  On sait qu'il existe P tel que $L = \{w \in \Sigma^* \mid \varphi(w) \in P\}$. Regardons ce qu'est $\varphi(L)$. On a que $\varphi(L) = \{\varphi(w) \mid w \in \Sigma^*, \varphi(w) \in P\} = P$. Et donc on a bien $\varphi^{-1}(\varphi(L)) = \varphi^{-1}(P) = L$.

□

Proposition 5.2. *Soit $L \subseteq \Sigma^*$ un langage sur l'alphabet Σ. Alors L est reconnaissable si et seulement si L est reconnu par un morphisme de monoïdes $\varphi : \Sigma^* \to M$ où M est fini.*

Démonstration.
- $\boxed{\Leftarrow}$
  Soit $\varphi : \Sigma^* \to M$ un morphisme de monoïdes avec M fini qui reconnait le langage L. $\exists P \subseteq M, L = \varphi^{-1}(P)$.
  Soit A l'automate suivant : $\langle M, 1_M, P, \delta \rangle$ où

$$\begin{array}{rcl} \delta : M \times \Sigma & \to & M \\ (m, a) & \mapsto & m \cdot \varphi(a) \end{array}$$

  On peut montrer par induction sur la longueur du mot que

$$\begin{array}{rcl} \delta^* : M \times \Sigma^* & \to & M \\ (1_M, w) & \mapsto & \varphi(w) \end{array}$$

$$\begin{array}{rcl} A \text{ accepte le mot } w & \iff & \delta^*(1_M, w) \in P \iff \varphi(w) \in P \\ & \iff & w \in \varphi^{-1}(P) \iff w \in L. \end{array}$$

Donc A accepte le langage L.

— $\boxed{\Rightarrow}$

Soit $A = \langle Q, q_0, F, \delta \rangle$ un automate déterministe complet qui accepte L. Soit

$$\begin{array}{rcll} \varphi : \Sigma^* & \to & Q^Q & \big((Q^Q, \circ, id_Q) \textit{ est bien un monoïde}\big) \\ w & \mapsto & (q \mapsto \delta^*(q, w)) & \end{array}$$

On a bien :

— Q^Q est un monoïde fini.

— $\varphi(\varepsilon)(q) = \delta^*(q, \varepsilon) = q$, donc $\begin{array}{rcl} \varphi(\varepsilon) : Q & \to & Q \\ q & \mapsto & q \end{array}$ est la fonction identité sur Q et donc φ préserve l'identité (élément neutre).

— $\varphi(w) \circ \varphi(w') = \varphi(ww')$, $\forall w, w' \in \Sigma^*$ car $\delta^*(\delta^*(q, w), w') = \delta^*(q, ww')$. Donc φ préserve la multiplication.

Soit $P \subseteq Q^Q$ défini par $P = \{f : Q \to Q \mid f(q_0) \in F\}$.

$$\begin{array}{rcl} \varphi^{-1}(P) & = & \{w \in \Sigma^* \mid \varphi(w) \in P\} \\ & = & \{w \in \Sigma^* \mid \varphi(w)(q_0) \in F\} \\ & = & \{w \in \Sigma^* \mid \delta^*(q_0, w) \in F\} \\ & = & \{w \in \Sigma^* \mid w \text{ est accepté par } A\} \\ & = & L \end{array}$$

Donc L est reconnu par φ.

□

Définition 5.5. On va appeler $\varphi(\Sigma^*) \subseteq Q^Q$ le **monoïde de transition de l'automate** A.

Définition 5.6 (Congruence)**.** Soit $(M, \cdot, 1_M)$ un monoïde. Une congruence sur M est une relation d'équivalence $\sim \subseteq M \times M$ telle que

$$\forall m, m' \in M,\ m \sim m' \iff \forall n, r \in M, n \cdot m \cdot r \sim n \cdot m' \cdot r$$

Définition 5.7 (Congruence syntaxique d'un langage)**.** Soit $L \subseteq \Sigma^*$ un langage. Soit $\sim_L \subseteq \Sigma^* \times \Sigma^*$ la relation d'équivalence définie par :

$$w \sim_L w' \text{ si et seulement si } \forall u, v \in \Sigma^*, uwv \in L \iff uw'v \in L$$

Proposition 5.3. *$\sim_L$ est une congruence sur Σ^*.*

Démonstration. Soit $m, m' \in \Sigma^*$ tels que $m \sim_L m'$, c'est-à-dire $\forall u, v \in \Sigma^*, umv \in L \iff um'v \in L$. Soit $n \in \Sigma^*$, montrons que $nmr \sim_L nm'r$ c'est-à-dire $\forall u, v \in \Sigma^*, u(nmr)v \in L \iff u(nm'r)v \in L$. Soit $u, v \in \Sigma^*$, posons $u' = un, v' = rv$, alors, on veut montrer que $u'mv' \in L \iff u'm'v' \in L$, ce qui est vrai car $m \sim_L m'$. □

Proposition 5.4. *Si $(M, \cdot, 1_M)$ est un monoïde et $\sim \subseteq M \times M$ est une congruence sur M, alors $M/\sim = \{[m] \mid m \in M\}$, où $[m]$ note la classe d'équivalence de m par rapport à $\sim$.*

$M/\sim$ est un monoïde et $\begin{array}{rcl} h : M & \to & M/\sim \\ m & \mapsto & [m] \end{array}$ *est un morphisme de monoïdes.*

Démonstration. $(M/\sim, \cdot, [1_M])$ est un monoïde où $[m][m'] = [mm']$.

À montrer que cette multiplication est bien définie, c'est-à-dire si $m_1 \sim m_2$ et $m_1' \sim m_2'$ alors $m_1 m_1' \sim m_2 m_2'$:

$$\begin{array}{rcll} m_1 & \sim & m_2 & \\ m_1' & \sim & m_2' & \\ 1_M \cdot m_1 \cdot m_1' & \sim & 1_M \cdot m_2 \cdot m_1' & (\textit{car} \sim \textit{ est une congruence}) \\ m_1 \cdot m_1' & \sim & m_2 \cdot m_1' & \\ \text{Mais } m_2 \cdot m_1' & \sim & m_2 \cdot m_2' & (\textit{car} \sim \textit{ est une congruence}) \\ \implies m_1 \cdot m_1' & \sim & m_2 \cdot m_2' & \end{array}$$

Donc la multiplication sur $M/\sim$ est bien définie.

$[1_M]$ est un élément neutre car $[m] \cdot [1_M] = [m \cdot 1_M] = [m]$ et pareil pour $[1_M] \cdot [m] = [m]$.

h est un morphisme de monoïdes :

— $h(1_M) = [1_M]$

— $h(m \cdot n) = [m \cdot n] = [m] \cdot [n] = h(m) \cdot h(n)$

□

Remarque 5.1.1. *h est un **morphisme surjectif** (aussi appelé un quotient).*

Définition 5.8. Soit $L \subseteq \Sigma^*$ un langage et soit $\sim_L$ la congruence syntaxique sur Σ^*. Le monoïde quotient $\Sigma^*/\sim_L$ est appelé le monoïde syntaxique de L.

Proposition 5.5. *Avec les notations ci-dessus, L est reconnu par le morphisme* $\begin{array}{rcl} \varphi : \Sigma^* & \to & \Sigma^*/\sim_L \\ w & \mapsto & [w] \end{array}$.

Remarque 5.1.2. *Si $w \in L$ et $w' \sim_L w$ alors $w' \in L$, parce que $w' \sim_L w$, donc $\varepsilon w \varepsilon \in L \implies \varepsilon w' \varepsilon \implies w' \in L$.*

Démonstration. Soit $P \subseteq \Sigma^*/\sim_L$ donné par $P = \{[w] \mid w \in L\}$.

On doit montrer que $L = \varphi^{-1}(P)$, c'est-à-dire , $L = \{w \in \Sigma^* \mid \varphi(w) \in P\}$

En utilisant la remarque 5.1.2, on a que si $w \in L$ alors $[w] \subseteq L$.

On en conclut que

$$L = \bigcup_{w \in L} [w] \iff L = \varphi^{-1}(P)$$

□

Proposition 5.6. *Le monoïde syntaxique a une propriété similaire à 4.2 : $\forall (M, \cdot, 1_M)$ monoïde qui reconnait un langage L, on a le diagramme suivant :*

$$\begin{tikzcd} & (N, \cdot, 1_N) \arrow[dl, hook] \arrow[dr, hook] & \\ (\Sigma^*/\sim_L, \cdot, [\varepsilon]) & & (M, \cdot, 1_M) \end{tikzcd}$$

Terminologie 5.1. *On dit que le monoïde syntaxique de L divise tout monoïde qui accepte L.*

Lemme 5.1. *Si N divise M et N reconnait un langage L alors M reconnait L.*

Démonstration.

$$\begin{tikzcd} & T \arrow[dl, two heads] \arrow[dr, hook] & \\ N & & M \end{tikzcd}$$

$\varphi : \Sigma^* \to N$ et $P \subseteq N$ tel que $L = \varphi^{-1}(P)$

Exercice : montrer que on peut construire $\varphi'' : \Sigma^* \to M$ qui reconnait L. □

Proposition 5.7. *Un monoïde M reconnait un langage L si et seulement si le monoïde syntaxique $\Sigma^*/\sim_L$ divise M.*

Démonstration.

— $\boxed{\Leftarrow}$

$\Sigma^*/\sim_L$ reconnait L. Par le lemme, si $\Sigma^*/\sim_L$ divise M, alors M reconnait L.

— $\boxed{\Rightarrow}$

Supposons que M reconnait L. Donc on a $\varphi : \Sigma^* \to M$ et $P \subseteq M$ avec $\varphi^{-1}(P) = L$.
Soit $N = \varphi(\Sigma^*)$ l'image directe de Σ^* par φ.
Alors $N \hookrightarrow M$ est un sous-monoïde de M.
Je voudrais avoir un quotient $h : N \twoheadrightarrow \Sigma^*/\sim$.
Si $n \in N$ alors $\exists w \in \Sigma^*$ tel que $\varphi(w) = n$. On définit $h(n) = [w]$.
Il faut montrer que :

1. h est bien défini (si $\varphi(w') = \varphi(w) \implies [w] = [w']$).
2. h est un morphisme de monoïdes.
3. h est surjectif.

Montrons cela :

— 1 : Supposons que $\varphi(w) = \varphi(w')$.

On veut montrer que $[w] = [w']$ c'est-à-dire $w \sim_L w'$, c'est-à-dire $\forall u, v \in \Sigma^*, uwv \in L \iff uw'v \in L$.

Soient $u, v \in \Sigma^*$.

$$\begin{aligned} uwv \in L &\iff \varphi(uwv) \in P \\ &\iff \varphi(u)\varphi(w)\varphi(v) \in P \quad (\textit{car } \varphi \textit{ est un morphisme}) \\ &\iff \varphi(u)\varphi(w')\varphi(v) \in P \quad (\textit{car } \varphi(w) = \varphi(w')\ ) \\ &\iff \varphi(uw'v) \in P \\ &\iff uw'v \in L \end{aligned}$$

Donc $w \sim_L w'$.

Exercice : finir la preuve.

□

6 Logique monadique du second ordre (MSO)

Est-ce qu'il existe $w \in \Sigma^*$ tel que $w \models \exists x \exists y (x < y)$ soit vraie ? (x, y sont interpretés comme des positions dans le mot w.) Par exemple, $a \not\models \exists x \exists y (x < y)$ Ici, cette proposition est satisfaite par $w \in \Sigma^*$ si et seulement si $|w| \geq 2$.

Pour une proposition φ, on a un langage $\mathcal{L}_\varphi = \{w \in \Sigma^* \mid w \models \varphi\}$.

Exemple 6.0.1. $\Sigma = \{a, b\}$ $\varphi_2 = \exists x \exists y (\forall z (z \geq x) \wedge Q_a x \wedge \forall z (z \leq y) \wedge Q_b y)$ où $\leq, \geq$ sont interpretés comme les relations habituelles sur N. $Q_a x$ signifie qu'en position x, on retrouve la lettre a. $\mathcal{L}_{\varphi_2} = \left\{ w \in \{a, b\}^* \mid w \models \varphi_2 \right\} = a(a + b) * b$

Exemple 6.0.2.

$$\begin{aligned} phi_3 &= \exists X (\forall x (\forall z (z \geq x) \rightarrow X(x))) \wedge \forall x (\forall z (z \leq x) \rightarrow \neg X(x)) \\ &\wedge \forall x \forall y (((x < y) \wedge \forall z (z > x) \rightarrow (z \geq y)) \rightarrow (X(x) \leftrightarrow \neg X(y))) \end{aligned}$$

$X \rightsquigarrow$ un ensemble de positions dans un mot
$X(x) \rightsquigarrow$ vrai si et seulement si $x \in X$
$\mathcal{L}_{\varphi_3} = \{w \in \Sigma^* \mid |w| \equiv 0 (\bmod 2)\}$

6.1 Syntaxe

Définition 6.1 (La logique du premier ordre)**.**
— **V** un ensemble de variables : $\{x, y, z, x_1, y_1, z_1, \ldots\}$
— Prédicats numériques : $\mathcal{P} = \left\{ R_i^j, i > 0, j \geq o \right\}$
— Formules atomiques :

$$\begin{array}{rcll} \alpha & ::= & Q_a x & \text{Lettre } a \text{ en position } x \\ & | & R_i^j(x_1, \ldots, x_j) & \text{Prédicats numériques} \end{array}$$

— Formules du premier ordre :

$$\begin{array}{rcll} \varphi & ::= & \alpha & \text{Formules atomiques} \\ & | & \varphi \wedge \varphi & \text{Conjonction} \\ & | & \neg \varphi & \text{Négation} \\ & | & \exists x \varphi & \text{Quantificateur existentiel} \end{array}$$

Définition 6.2 (La logique du second ordre)**.** C'est une extension de la logique du premier ordre :
— un ensemble de variables du second ordre : $X, Y, Z, X_1, Y_1, Z_1 \cdots$
— Formules atomiques :

$$\begin{array}{rcll} \alpha & ::= & \cdots & \\ & | & X(x) & \text{Appartenance à } X \end{array}$$

— Formules du second ordre :

$$\begin{array}{rcll} \varphi & ::= & \cdots & \\ & | & \exists X \varphi & \text{Quantificateur existentiel sur des ensembles} \end{array}$$

Notation 6.1 (Quantificateur universel)**.** On note $\forall x \varphi$ la formule $\neg \exists x (\neg \varphi)$. Cette notation s'étend aussi à la logique du second ordre.

6.2 Sémantique

6.2.1 Sémantique de la logique monadique du premier ordre

Pour $n \in \mathbb{N}$ la longueur d'un mot, x est interpreté comme un élément de $\{1, \cdots, n\}$

$$[\![R_i^j]\!]_n \subseteq \{1, \cdots, n\}^j$$

est une interpretation de chaque prédicat R_i^j pour tout $n \geq 1$.

$$[\![<]\!]_n \subseteq \{1, \cdots, n\} \times \{1, \cdots, n\} = \{(i, j) \mid i < j\}$$

Définition 6.3. Une $\mathcal{V}$-structure, pour $\mathcal{V} \subseteq \mathbf{V}$ est un mot de la forme $(a_1, U_1) \dots (a_n, U_n)$ où $a_1, \dots, a_n \in \Sigma$ et $U_1, \dots, U_n \subseteq \mathbf{V}$ tels que

1. $U_i \cap U_j = \emptyset, \forall i, j,\ i \neq j$
2. $\bigcup_{i=1}^n U_i = \mathcal{V}$

Définition 6.4 (relation de satisfaction)**.** On définit la relation de satisfaction $w \models \varphi$ où w est une $\mathcal{V}$-structure et φ une formule de premier ordre tel que

1. $\forall x \in \mathrm{FV}(\varphi) \implies x \in \mathcal{V}$ (FV = ensemble des variables libres)
2. Les quantificateurs distincts dans φ lient des variables distinctes

Si φ est un proposition, c'est-à-dire , $FV(\varphi) = \emptyset$ alors on a $\mathcal{L}_\varphi = \{w \text{ les } \emptyset\text{-structures} \mid w \models \varphi\} \subseteq \Sigma^*$.

Pour une interprétation $[\![R_i^j]\!]_{i,j}$ de prédicats numériques, on définit la relation $w \models \varphi$ par induction sur la structure de la formule φ.

— $w \models Q_a x$ ssi w contient une lettre (a, U_i) avec $x \in U_i$
— $w \models R_i^j(x_1, \dots, x_j)$ ssi $[\![R_i]\!]_{|w|}(k_1, \dots, k_j)$ est vrai où les $k1, \dots, k_j$ sont les positions dans w où les variables $x_1, \cdots x_j$ apparaissent.
— $w \models \varphi_1 \wedge \varphi_2$ ssi $w \models \varphi_1$ et $w \models \varphi_2$
— $w \models \neg\varphi$ ssi $w \not\models \varphi$
— si $w = (a_1, U_1) \dots (a_n, U_n)$ est une $\mathcal{V}$-structure. $w \models \exists\varphi$ ssi $\exists i \in \{1, \dots, n\}$ $(a_1, U_1) \dots (a_i, U_i \cup \{x\}) \dots (a_n, U_n) \models \varphi$ (il s'agit d'une $\mathcal{V} \cup x$-structure.)

Exemple 6.2.1.

$$abc \models_? \exists x \exists y (x < y)$$

$(a, \{x\})(b, \emptyset)(c, \emptyset) \models_? \exists y (x \leq y)$ est une $\{x\}$-structure
$(a, \{x\})(b, \{y\})(c, \emptyset) \models_? (x < y) \iff (1, 2) \in [\![<]\!]_3 \iff 1 < 2$
Version 2

Soit φ et ψ deux formules, $\varphi \iff \psi$ si et seulement si $\mathcal{L}_\varphi = \mathcal{L}_\psi$.

Abbréviation 6.1. $\forall x \varphi = \neg \exists x (\neg \varphi)$

Exemple 6.2.2. TODO : add it (important)

6.2.2 Sémantique de la logique monadique du second ordre

V_1 les variables du premier ordre
V_2 les variables du second ordre

Définition 6.5. Une $(\mathcal{V}_1, \mathcal{V}_2)$-structure est un mot sur $\Sigma \times 2^{\mathcal{V}_1} \times 2^{\mathcal{V}_2}$, c'est-à-dire un mot de la forme $(a_1, U_1, U_1) \cdots (a_n, U_n, V_n)$ où

— Les U_i sont des ensembles de variables du premier ordre.
— Les V_i sont des ensembles de variables du second ordre.
— $(a_1, U_1) \cdots (a_n, U_n)$ est une $\mathcal{V}_1$-structure.

Définition 6.6. On définit la relation de satisfaction $w \models \varphi$ où w est une $(\mathcal{V}_1, \mathcal{V}_2) - structure$ et φ une formule de MSO avec les mêmes contraintes sur les variables que pour la logique du premier ordre plus $w \models X(x)$ et $w \models \exists X \varphi$

L'induction est la même que pour le premier ordre, avec ces cas en plus :
— $w \models X(x)$ ssi w contient une lettre (a, S, T) où $x \in S$ et $X \in T$
— $w \models \exists X \varphi$ ssi $\exists J$ un ensemble de positions dans w avec la propriété : la $(\mathcal{V}_1, \mathcal{V}2)$-structure w' obtenue en remplaçant (a_i, S_i, T_i) pour $i \in J$ par $(a_i, S_i, T_i \cup \{X\})$ satisfait φ.

Exemple 6.2.3. TODO

Définition 6.7. Un langage L est définissable dans MSO[$<$] (la logique monadique du second ordre avec un prédicat binaire $<$). si et seulement si il existe une formule $\varphi \in MSO[<]$ telle que $L = \{w \in \Sigma^* \mid w \models \varphi\}$

Définition 6.8. Un langage $L \subseteq \Sigma^*$ est définissable dans $FO[<]$ (la logique du premier ordre avec $<$) si et seulement si $\exists \varphi \in FO[<]$ telle que $L = \{w \in \Sigma^* \mid w \models \varphi\}$

6.3 Relation avec les expressions rationnelles

Théorème 6.1. *Un langage est définissable dans $MSO[<]$ ssi L est régulier.*

Démonstration. — $\boxed{\Leftarrow}$

Soit $A = \langle Q, q_0, F, \delta \rangle$ un automate fini déterministe qui accepte un langage L. On peut supposer que $L \subseteq \Sigma^*$ (car $\varepsilon \in L$, on va prendre la disjonction de la formule obtenue pour $L \setminus \{\varepsilon\}$ avec $\forall x \neg(x = x)$)

Soit $w \in \Sigma^*$, Alors w est reconnu par l'automate A ssi $\exists X_0, \cdots, X_{k-1} \subseteq \{1, \cdots, |w|\}$ tels que les propriétés suivantes soient vérifiés :

1. $\bigcup_{i=0}^{k-1} X_i = \{1, \cdots, |w|\}$
2. $\forall i < j, X_i \cap X_j = \emptyset$
3. $1 \in X_0$
4. $\forall j \in \{1, \cdots, |w|\}$ si $j \in X_i$ et $j + i \in X_e$ et si a est la lettre en position j dans le mot w, alors $\delta(q_i, a) = q_e$.
5. Si $|w| \in X$ et a est la dernière lettre de w, alors $\delta(q_j, a) \in F$.

Supposons que $w = a_1 a_2 \ldots a_n$ est accepté par A. On construit les ensembles $(X_i)_{i \in \{0, \ldots, k-1\}}$ tel que $i \in X_j$ ssi après avoid lu les premiers $i - 1$ lettres de w on arrive a l'état q_j

Les ensembles $X_0, \cdots, X_{k-1}$ satisfait les 5 propriétés.

1. $X_0 \cup \cdots \cup X_{k-1} = \{i, \ldots, |w|\}$. L'inclusion a gauche est vraie par définition. Montrons l'autre inclusion. On considère l'unique chemin dans l'automate A obtenu en lisant les premieres $i - 1$ lettres de w à partir de l'état q_0. Supposons qu'on arrive dans l'état q_j. Par définitions $u \in X_j$.
2. $\forall i < j, X_i \cap X_j = \emptyset$ est vrai car l'automate est déterministe.

3. $1 \in X_0$ car si on lit les premieres 0 lettres de w on reste a l'état q_0.
4. Après les premieres $j-1$ lettres on arrive a q_i. a est la lettre en position j. Donc après les premieres j lettres on arrive dans l'état $\delta(q_i, a)$. Si après les premières j lettres on arrive dans l'état q_l, alors $\delta(q_i, a) = q_l$.
5. w est accepté par A. Donc si $|w| \in X_j$ alors après avoir lu les premieres $|w|-1$ lettres on arrive dans q_j. Si a est la dernière lettre de w, alors $\delta(q_j, a)$ est un tat acceptant car w est accepté.

On construit la formule

$$\varphi = \exists X_0 \cdots \exists X_{k-1}(\varphi_1 \wedge \cdots \wedge \varphi_5)$$

où

— $\varphi_1 = \forall x(X_0(x) \vee \cdots \vee X_{k-1}) = \forall x \bigvee_{i=1}^{k-1} X_i(x)$
— $\varphi_2 = \forall x \bigwedge_{0 \leq i < j \leq k-1} \neg(X_i(x) \wedge X_j(x))$
— $\varphi_3 = \exists x\, ((\forall y, x \leq y) \wedge X_0(x))$
— $\varphi_4 = \forall x \left(\forall y(y = x+1) \rightarrow \bigwedge_{0 \leq i < l < k}((X_i(x) \wedge X_l(y)) \rightarrow \bigvee_{S_l} Q_a(x))\right)$
Où $S_l = \{a \in \Sigma \mid \delta(q_i, a) = a_l\}$
— $\varphi_5 = \forall x \left(\forall y(x \geq y) \rightarrow \bigwedge_{i=0}^{k-1}(X_i(x) \rightarrow \bigvee_{T_i} Q_a(x))\right)$
Où $T_i = \{a \in \Sigma \mid \delta(q_i, a) \in F\}$

— $\boxed{\Rightarrow}$

On suppose que L est défini par une formule de $MSO[<]$. La preuve est par induction sur la structure de la formule.

$L \subseteq (\Sigma \times 2^{\mathcal{V}_1} \times 2^{\mathcal{V}_2})$.

$\mathcal{L}$ est l'ensemble de toutes les $(\mathcal{V}_1, \mathcal{V}_2)$-structures, pour $\mathcal{V}_1$ en ensemble de variables du premier ordre et $\mathcal{V}_2$ un ensemble de variables du second ordre.

Exercice 6.3.1. Trouver un automate sur l'alphabet $\Sigma \times 2^{\mathcal{V}_1} \times 2^{\mathcal{V}_2}$ qui accepte $\mathcal{L}$

Exercice 6.3.2. Trouver un automate qui accepte les $(\mathcal{V}_1, \mathcal{V}_2)$-structures tel qu'une variable de premier ordre x apparait dans une lettre de la forme (a, S, T). Le but est de montrer que le $\mathcal{L}(Q_a(x))$ est régulier.

Exercice 6.3.3. Montrer que $\mathcal{L}_{X(x)}$ est régulier.

Exercice 6.3.4. Montrer que $\mathcal{L}_{x<y}$ est régulier.

Soit $\varphi = \exists x \psi$ et supposons que L_ψ est régulier et donc accepté par un automate $A = \langle Q, q_0, F, \delta \rangle$ sur l'alphabet $\Sigma \times 2^{\mathcal{V}_1} \times 2^{\mathcal{V}_2}$.

On définit l'automate $A' = \langle Q \times \{0,1\}, (q_0, 0), F \times \{1\}, \delta' \rangle$ où

$$\begin{aligned} \delta' &= \{((q,u),(a,S,T),(q',u)) \mid u \in \{0,1\}, x \notin S, (q,(a,S,T),q') \in \delta\} \\ &\cup \{((q,0),(a,S \setminus \{x\},T),(q',1)) \mid x \in S, (q,(a,S,T),q') \in \delta\} \end{aligned}$$

A' est un automate sur l'alphabet $\Sigma \times 2^{\mathcal{V}_1 \setminus \{x\}} \times 2^{\mathcal{V}_2}$.

□

Théorème 6.2. *Un langage est définissable dans $FO[<]$ ssi il existe une expression rationnelle sans étoile r définissant un langage ssi le monoïde syntaxique du langage est apériodique (c'est-à-dire il ne contient aucun sous-groupe non-trivial).*

6.3.1 Éléments idempotents

Définition 6.9 (Éléments idempotents)**.** Soit $(M, \cdot, 1)$ un monoïde. Un élément $e \in M$ est *idepmpotent* si et seulement si $e \cdot e = e$.

Exercice 6.3.5. Soit $e \in M$ est idempotent, alors $\forall i \in \mathbb{N}^*$. $e^i = e$.

Lemme 6.1. *Soit $(M, \cdot, 1_M)$ un monoïde fini et soit $m \in M$. Si $|M| = n$ alors $m^{n!}$ est idempotent.*

Démonstration. Parmi les puissances de m : $m, \cdots, m^{n+1}$ il y deux qui sont égales, car $|M| = n$. Soient $k \in \{1, \ldots, n\}$ et $l \geq 1$ tel que $m^k = m^{k+l}$ $(k + l \in \{1, \ldots, n+1\})$. $m^k = m^{k+l} \implies m^{k+2l} = m^{k+l}m^l = m^k m^l = m^{k+l} = m^k$

Par induction on a que $\forall i \geq m^{k+il} = m^k \implies \forall i \geq 1, j \geq 1, m^{k+il+j} = m^{k+j}$

Donc $\exists x \in \mathbb{N}$, $m^{n!} = m^{klx} = (m^{kl})^x = m^{kl} \implies m^{n!}$ est idempotent. □

Définition 6.10 (Puissance idempotente)**.** Une *puissance idempotente* d'un élément $m \in M$ est un élément de la forme m^i $(i \geq 1)$ tel que m^i est idempotent.

Lemme 6.2. *Une puissance idempotente de m in$M$ es unique, dans le sense suivant. Si m^i et m^j sont des idempotents, alors $m^i = m^j$.*

Démonstration. m^i idempotent $\implies (m^i)^j = m^i$, voire 6.3.5.

Pareil pour m^j idempotent $\implies (m^j)^i = m^j$ Donc $m^i = m^{ij} = m^{ji} = m^j$ □

6.3.2 Relations de Green

Définition 6.11. Soit $(M, \cdot, 1_M)$ un monoïde. On défini les relations suivantes.

— *prefixe, ou $>_R$, ou R-préordre* : On dit que m est un prefixe de n si et seulement si $\exists x \in M,\ n = mx$ On va noter cela par $m >_R n$.

Un ensemble de la forme $y \cdot M = \{y \cdot z \mid z \in M\}$ pour $y \in M$ est appelé un idéal à droite de M.

m est un préfixe de n ssi $m \cdot M \supseteq n \cdot M$ (à vérifier en exercice)

— *suffixe, ou $>_L$, ou L-préordre* : On dit que m est un suffixe de n ssi $\exists y \in M,\ n = ym$ On va noter cela par $m >_L n$.

TODO : exercice (same as before)

— *infixe, ou $>_J$, ou J-préordre* : On dit que m est un infixe de n ssi $\exists x, y \in M,\ n = xmy$ On va noter cela par $m >_J n$.

TODO : exercice (same as before)

Exercice 6.3.6. Montrer que $>_R, >_J, >_L$ sont transitives et réflexives et donc des préordres.

Notation 6.2. On défini les relations d'équivalence associées à ces préordres L

— mRn ssi $m >_R n$ et $n >_R m$ (On dit m est R-équivalent n)

On note par $[m]_R$ la R-classe d'équivalence de m, *i.e.* , $[m]_R = \{n \in M \mid mRn\}$

— mLn ssi $m >_L n$ et $n >_L m$ (On dit m est L-équivalent n)

On note par $[m]_L$ la L-classe d'équivalence de m, *i.e.* , $[m]_L = \{n \in M \mid mLn\}$

— mJn ssi $m >_J n$ et $n >_J m$ (On dit m est J-équivalent n)

On note par $[m]_J$ la J-classe d'équivalence de m, *i.e.* , $[m]_J = \{n \in M \mid mJn\}$

Exercice 6.3.7. $\forall m \in M, [m]_R \subseteq [m]_J$

Démonstration. TODO □

Lemme 6.3 (Eggbox lemma)**.** *Soit M un monoïde fini et $m, n \in M$,*

1. *Si $m >_R n$ et mJn alors $n >_R m$ et donc mRn*
2. *Si $m >_L n$ et mJn alors $n >_L m$ et donc mLn*

Démonstration.

1. Soit $m, n \in M$ tels que $m >_R n$ et mJn alors

$$m >_R n \implies \exists x \in M,\ n = mx$$

$$mJn \implies \exists y, z \in M,\ m = ynz$$

Tout ça implique $m = ymxz$

TODO

2. Exercice

□

Définition 6.12. On dit mHn ssi mRn et mLn. On a $H = R \cap L$.

Exercice 6.3.8. L'intersection d'une R-classe avec une L classe d'équivalence est soit vide soit une H-classe d'équivalence.

Démonstration. Debut de preuve : Si $[m]_R \cap [n]_L \neq \emptyset$, sot $s \in [m]_R \cap [n]_L$. On va montrer que $[m]_R \cap [n]_L = [s]_H$

— Soit $s' \in [m]_R \cap [n]_L$. On sait $s'Rs$ car $s' \in [m]_R$ donc $s'RmRs$. Pareil, $s'Ls$ car $s'LbLs$. Et donc $s'Hs$. Et donc $[m]_R \cap [n]_L \subseteq [s]_H$

—

$$\begin{aligned} s' \in [S]_H &\implies sRs' \text{ et } sLs' && (2)\\ &\implies s'Rm \text{ et } s'Ln && (3)\\ &\implies s' \in [m]_R \cap [n]_L && (4) \end{aligned}$$

□

Lemme 6.4. *Soit $\mathbb{H}$ une H-classe incluse dans une J-classe $\mathbb{J}$. Alors, soit*

1. *$\forall m, n \in \mathbb{H}, mn \notin \mathbb{J}$*
2. *$\mathbb{H}$ est un groupe (Attention !! L'élément neutre de $\mathbb{H}$ n'est pas forcement 1_M)*

Démonstration. TODO

— Etape 2.

Exercice 6.3.9. Montrer que $mnLn$.

— Étape 3 : Soit $m \in \mathbb{H}$. Par 6.2, il existe $i > 1$ tel que m^i est idempotent. Par l'etape 3, on sait que $m^i \in \mathbb{H}$, donc on peut conclure que $\mathbb{H}$ contient un élément idempotent, notos le par e. On va montrer que e est l'élément neutre de $\mathbb{H}$, c'est-à-dire , $\forall m \in \mathbb{H},\ m \cdot eme \cdot m = m$.

Soit $n \in \mathbb{H}$, donc $nLe \implies \exists x,\ n = x \cdot e \implies ne = (xe)e = x(ee) = xe = n$

De la même maniere, $n \in \mathbb{H} \implies nRe \implies \exists y,\ n = ey \implies en = eey = ey = n$ Donc $en = n$.

Il reste a montrer que chaque élément possède un inverse : $\forall n \in \mathbb{H}, \exists n' \in \mathbb{H}$ tel que $nn' = e = n'n$

On observe que $\mathbb{H}$ contient un unique idempotent. Soit e' un autre idempotent, par le même argument que ci-dessus, e' es un élément neutre donc : TODO ... $e =' e$

Soit $n \in \mathbb{H}$. Par le lemme 6.2m il existe $j \geq 1$ tel que n^j est idempotent, mais $n^j \in \mathbb{H}$ donc $n^j = e$. Et ainsi, n^{j-1} est l'inverse de n.

□

Définition 6.13. Un monoïde $(M, \cdot, 1_M)$ est aperiodique ssi $\forall m \in M, \exists i \geq 1,$ tel que $m^i = m^{i+1}$

Théorème 6.3 (Schützenberger)**.** *Soit $L \subseteq A^*$ un langage régulier. Les affirmations suivantes sont équivalentes :*

1. *L est définissable par une expression rationnelle sans-étoile.*
2. *L est définissable dans la logique $FO[<]$.*
3. *Le monoïde syntaxique de L est apériodique.*

TODO

Références

[David(2010)] Julien David. The average complexity of moore's state minimization algorithm is o(n log log n). In *International Symposium on Mathematical Foundations of Computer Science*, 2010. URL https://api.semanticscholar.org/CorpusID:9900076.